

문제를 푸는 방식

이기춘 — AI R&D 엔지니어 · 모델 학습 · 도메인 에이전트 · AI 프로토타이핑

현재 역할 AI 파트 리드 (직급 대리) 병역 전문연구요원 복무 만료	소속 (주)올포랜드 공간융합연구소 연락 accforaus@gmail.com	학위 공학석사 · 군산대 컴퓨터정보공학
상시 서빙 모델 8종 전사 단일 API로 통합	가동 서비스 15개 사내 AI 플랫폼이 수용	경력 3년 3개월 2023.05 ~ 현재
역할 기술 리드 AI 파트 · 7명 규모	수행 과제·사업 8건 공공SI 3 · 국책R&D 4 · 장기 1	운영 GPU 19대 서빙 15 · 학습 전용 4

WHO AM I

주어진 문제를 푸는 것보다, **아직 정의되지 않은 문제를 찾아 검증 가능한 형태로 바꾸는 일을 더 많이 해왔습니다.**

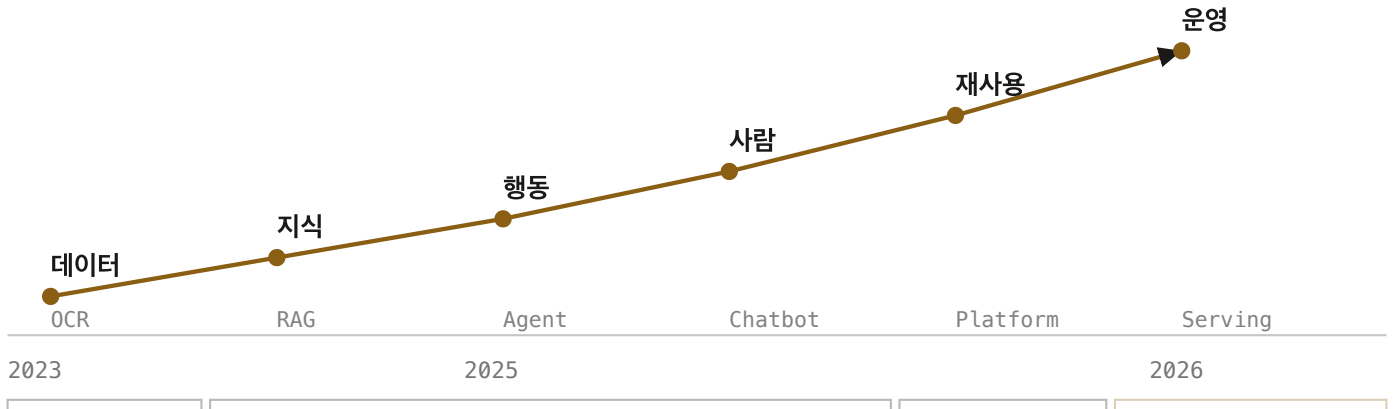
- 전사 AI 기반을 만들고 운영**
인증·인가를 게이트웨이로 강제하는 사내 AI 플랫폼을 설계해 **서비스 15개**를 수용했고, 사내·임차 GPU 15대 위에서 **모델 8종**을 단일 API로 서빙합니다.
- 외부 솔루션 없이 자체 OCR 모델 확보**
손글씨 한자 고문서에서 사전학습 소스가 덮지 못한 **208자**를 생성 모델로 보강해 **H-mean 0.8203**을 만들었습니다.
- 과제가 아니던 것을 과제로 만들**
사내 챗봇 · AI 플랫폼 · GPU 벤치마크 · 코드 어시스턴트 — **사내 AI 제품은 전부 제가 발의해 시작했습니다.** 구현까지 직접 한 것과, 설계만 잡고 팀에 넘긴 것을 구분해 적었습니다.

기술을 고르는 일보다 무엇이 진짜 병목인지 알아내는 데 시간을 더 씁니다. **대개 병목은 처음 짐작한 곳에 없었습니다.** 아래 여섯 장은 그 짐작이 어디서 어떻게 틀렸는지에 대한 기록입니다.

3년 동안 매년 다루는 단위가 한 단계씩 커졌습니다. 데이터에서 시작해 지식, 행동, 사람, 재사용, 운영으로 옮겨 갔습니다. 각 단계는 앞 단계에서 **틀렸던 판단** 때문에 시작됐습니다.

이 문서는 프로젝트 목록이 아니라 그 판단들의 기록입니다. 각 장은 문제 → 판단 → 구현 → 결과 → 배운 점 순으로 되어 있고, 배운 점은 다음 장의 문제로 이어집니다.

성장 곡선



위쪽 축은 다루는 대상이 커진 순서, 아래쪽 떠는 다루는 계층입니다. 모델을 만들다가, 그 모델을 쓰는 서비스를, 다시 그 서비스들을 떠받치는 기반을 맡게 됐습니다.

Chapter 1

데이터를 이해했다

고문서 OCR · 2023.05 – 2024.08 · AI 모델 단독 담당 **AI MODEL**

문제

지자체 구 토지대장을 인식하는 OCR. 조건이 겹쳐 있었습니다.

손글씨 한자 → 노후 문서라 노이즈가 심함 → 빈도 낮은 글자는 학습 데이터 자체가 없음

처음 가설과 그 한계

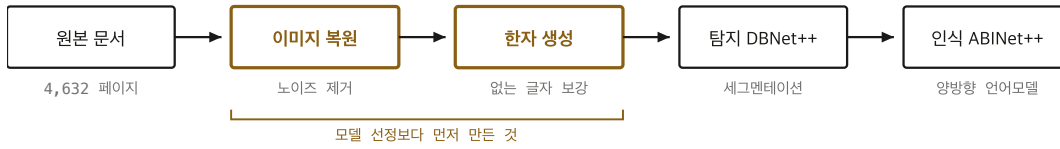
처음에는 탐지·인식 모델의 성능을 올리는 문제로 봤습니다. 노이즈가 심하니 이미지를 복원해 넣으면 되겠다고 판단했고, 실제로 복원 모델을 만들었습니다.

그런데 희귀 한자는 여전히 학습되지 않았습니다. 입력을 아무리 깨끗하게 만들어도, 애초에 데이터에 몇 번 나오지 않는 글자는 모델이 배울 수가 없었습니다.

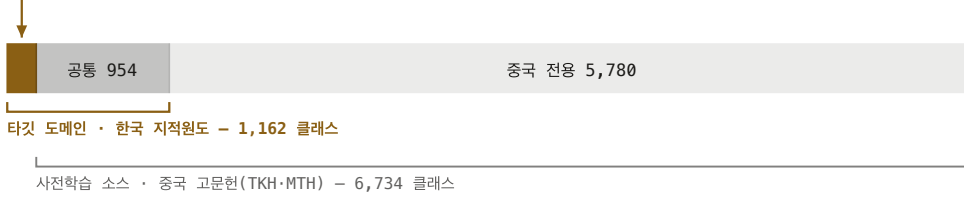
판단

선행 연구가 왜 실패했는지 항목화해 보니 성능 자체보다 평가 불가 · 전처리 설명 부족 · 후처리 미흡이 반복되고 있었습니다. 제가 겪은 것과 같은 문제였습니다.

가설을 바꿨습니다. 병목은 데이터의 양이나 깨끗함이 아니라 분포다. 없는 글자는 만들어서 채워야 한다.



한국 전용 208자 - 사전학습 소스에 없던 글자

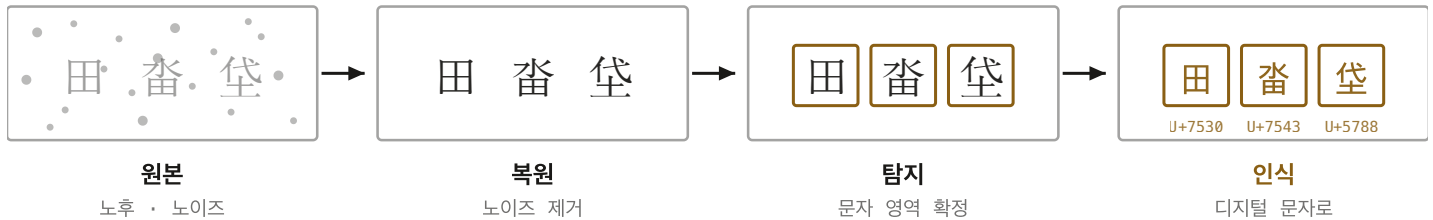


소스는 타깃의 6배 클래스를 덮지만, 정작 필요한 208자는 덮지 못한다 - 양의 문제가 아니라 분포의 문제

위는 만든 순서, 아래는 그렇게 만든 이유입니다. 사전학습 소스가 타깃보다 클래스 수가 **6배 많은데도** 208자가 비어 있었습니다. 데이터를 더 넣는 방향으로만 달지 않는 칸이라 생성 모델로 채웠습니다.

구현

모델 선정은 마지막이었습니다. 곡선형·비정형 영역이 많아 박스 기반보다 세그멘테이션 기반 탐지가 맞다고 보고 DBNet 계열을, 문맥으로 글자를 보정할 수 있는 양방향 반복 언어 모델링 구조가 필요해 ABINet 계열을 골랐습니다. AI 기반 구축지원시스템의 UI 설계서와 처리 흐름도까지 작성해 실제 작업자가 쓰는 도구로 연결했습니다.



※ 개념도입니다 - 실제 지자체 문서 이미지는 포함하지 않았습니다

각 단계가 이미지를 어떻게 바꾸는지입니다. 앞의 두 칸(복원·생성)이 **모델을 고르기 전에** 해결해야 했던 부분이고, 뒤의 두 칸이 흔히 말하는 OCR입니다.

결과

사전학습에 없던 문자 208자 이 칸을 메우는 게 과제였다	H-MEAN 0.8203 ICDAR 평가 프로토콜	학습 이미지 30,000+ 직접 구축 · 레이블링
정밀도 / 재현율 0.91 / 0.75 비대칭이 다음 과제가 됐다	레이블 문자 131만 자 한국 23만 · 중국 108만	적용 기관 3곳 지자체 구 토지대장

외부 솔루션 없이 자체 OCR 모델을 확보했고, 병역 주요업무 확인서에 "본인이 아니면 대행할 수 없는 업무"로 기재됐습니다.

정밀도 0.91에 비해 재현율 0.75는 낮습니다. 찾은 글자는 맞는데 아직 못 찾는 글자가 남아 있다는 뜻이고, 그 격차가 이후 고도화의 출발점이었습니다. 평균값 하나만 보고 있었다면 이 비대칭은 보이지 않았을 겁니다.

배운 점

데이터를 늘리는 것, 깨끗하게 만드는 것, 분포를 메우는 것이 전부 다른 일이라는 걸 이때 알았습니다. 복원 모델은 두 번째까지만 해결해 줬습니다.

지금도 새 과제를 시작하면 모델보다 데이터 분포를 먼저 봅니다. 어떤 클래스가 몇 건인지, 꼬리가 얼마나 긴지부터 세고 시작합니다. 다음 장이 그 습관에서 시작합니다.

Chapter 2

지식을 이해했다

RAG 서비스 · 2025 · 공공 사업 AI 파트 책임

AI SERVICE

문제

회사의 첫 LLM 서비스였습니다. 단순 챗봇이 아니라 비정형 문서(PDF), 정형 데이터(DB), 공간 데이터가 함께 결합된 형태가 요구됐습니다. 문서는 길고 형식이 제각각이었습니다.

판단

앞 장의 습관대로 데이터부터 봤습니다. 문제는 **검색이 문서 단위로 일어난다**는 것이었습니다. 긴 문서 하나가 통째로 후보가 되니 정작 필요한 문단이 묻혔습니다.

가설: 단위를 문서가 아니라 의미로 바꿔야 한다.

처음 가설과 실패한 순서

chunking만 바꾸면 해결될 줄 알았습니다. 단위가 문제라고 봤으니 단위만 손보면 된다고 본 것입니다. 그런데 성능이 거의 오르지 않았습니다.

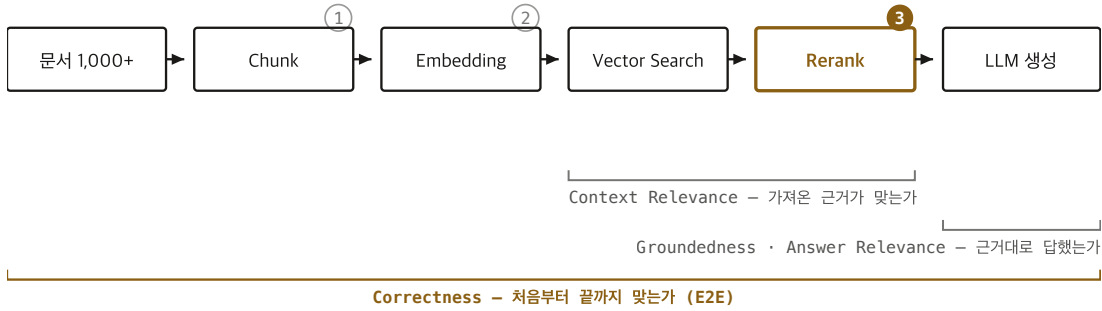
chunking 전략 변경 → 거의 변화 없음

임베딩 모델 교체 → 조금 오르다 멈춤

reranking 추가 → 여기서 올라감

세 단계가 각각 다른 이유로 성능을 깎고 있었습니다. 단계별로 분리해 측정하기 전까지는 어디가 문제인지 몰랐고, 그때까지 저는 계속 엉뚱한 손잡이를 돌리고 있었습니다.

시도한 순서 ① chunking 전략 변경 - 변화 없음 ② 임베딩 모델 교체 - 조금 오르다 멈춤
③ reranking 추가 - 여기서 올라갔다



평가 질의 200건 · 대상 문서 1,000+ · 개선 전후 동일 인덱스 / 동일 하드웨어

단계를 나눠 재기 전까지는 ①②③ 중 무엇이 듣는지 알 수 없었다

지표를 구간별로 붙여 두면 어느 단계를 건드렸을 때 어느 숫자가 움직이는지가 대응됩니다. 이 대응이 없던 동안 저는 ①과 ②에 시간을 썼습니다.

결과

검색 성능

40% ↑

chunking + reranking

검색 속도

3×

벡터 인덱싱 튜닝

추론 비용

50% ↓

동적 배치 처리

평가 질의 200건 · 대상 문서 1,000+ · 개선 전후 동일 인덱스 / 동일 하드웨어. 지표는 검색(Context Relevance) · 생성(Groundedness · Answer Relevance) · E2E(Correctness) 4축으로 나눠졌습니다.

요구사항 분석·기능 범위 설정·우선순위 결정까지 맡았습니다. AI 기능에 대한 불명확한 기대를 기술적 근거로 조정해 일정 지연을 사전에 차단했습니다.

배운 점

파이프라인은 단계별로 측정할 수 있게 만들어 두지 않으면 고칠 수 없습니다.

지금은 파이프라인을 만들 때 각 단계의 입출력을 먼저 짚어 볼 수 있게 해 줍니다. 기능보다 계측을 먼저 붙이는 편입니다.

그리고 다음 문제가 보였습니다. 검색은 됐는데, 사용자가 원하는 건 답이 아니라 동작이었습니다.

Chapter 3

행동을 이해했다

도메인 특화 에이전트 · 2025 · AI 파트 설계·구현

AI SERVICE

문제

디지털트윈 플랫폼에 AI를 붙이는 과제였습니다. 사용자가 원하는 것은 설명이 아니라 지도를 움직이고 객체를 찾는 것이었습니다. 질문에 답하는 것으로는 부족했습니다.

처음 만든 구조와 그 문제

처음에는 LLM이 해석부터 실행까지 맡는 구조로 만들었습니다. 그때는 재현성보다 개발 속도가 중요하다고 생각했습니다. 모델 하나에 전부 맡기면 붙이는 코드가 훨씬 적었습니다.

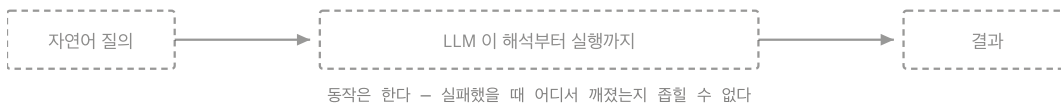
동작은 했습니다. 그런데 실패했을 때 어디서 실패했는지 짐힐 수 없었습니다. 같은 입력에 다른 결과가 나오니 버그인지 모델 특성인지도 구분되지 않았습니다. 빨리 만든 대신 고칠 수가 없었습니다.

판단

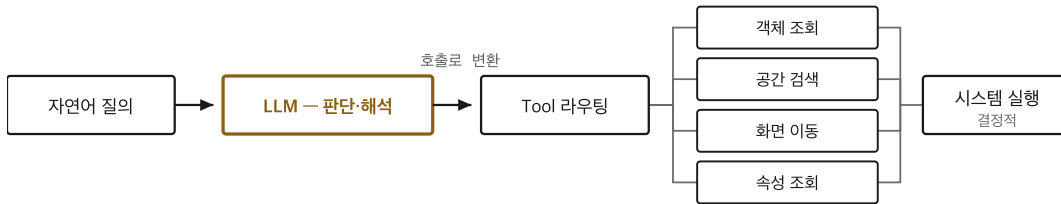
확률적인 부분을 없앨 수는 없으니, 어디에 가돌지를 정해야 한다고 봤습니다.

LLM은 판단과 해석만 하고, 실제 처리는 시스템이 한다. 그러면 실행은 결정적이 되고, 실패 시 어느 Tool에서 깨졌는지 특정됩니다.

처음 구조



바꾼 구조 - 확률적인 것을 한 칸에 가둔다



실패하면 어느 Tool에서 깨졌는지 특정된다 - 같은 입력이면 같은 결과

Tool 10종 이상 · 결합 시나리오 30건 이상 (행정 · 복지 · 위치 데이터 결합)

두 구조의 차이는 화살표 하나입니다. LLM 뒤로 결정적인 실행 계층을 세우자 디버깅 가능 여부가 갈렸습니다.

구현

LangChain / LangGraph 기반 에이전트로, 자연어를 객체 조회 · 검색 · 화면 이동 · 속성 조회 Tool 호출로 변환했습니다. 성격이 다른 데이터(행정·복지·위치)를 결합한 시나리오를 구성하고, 프론트엔드 담당자와 기술 경계·인터페이스 기준을 합의했습니다.

연결 TOOL

10+

조회 · 검색 · 화면 · 속성

처리 시나리오

30+

이종 데이터 결합

결합 데이터

3계열

행정 · 복지 · 위치

배운 점

확률적인 것을 어디에 가돌지가 설계의 핵심이라는 것. 속도를 위해 경계를 흐리면 결국 더 느려진다는 것도 같이 알았습니다.

지금은 LLM을 붙일 때 "이 출력이 틀리면 어디를 보면 되는가"를 먼저 정합니다. 답이 안 나오면 구조를 다시 나눕니다. 이 기준을 문서로 정리해 이후 과제에서 재사용하게 했습니다.

다만 이 기준은 지도가 기다려 주는 도메인 위에서 만들어진 것입니다. 사용자 상태가 계속 변하거나 지연시간 자체가 경험인 도메인이라면 Tool 경계를 처음부터 다시 그어야 한다고 봅니다.

Chapter 4

사람들이 쓰게 만들었다

사내 챗봇 · 2025 – 현재 · 발의 · 설계 · 구현 · 리드

스스로 정의한 문제

AI SERVICE

문제

기술은 세 장에 걸쳐 준비됐는데, 정작 **사내에서 쓰이는 AI는 없었습니다**. 지식은 그룹웨어·문서·메일로 흩어져 있었고 아무도 이걸 과제로 올리지 않았습니다. 지시받은 일이 아니라 **직접 기획안을 써서 상실했습니다**.

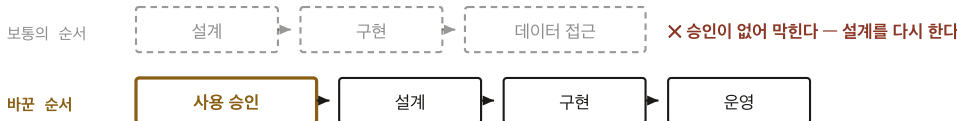
처음 가설과 그 한계

앞의 세 장에서 쌓은 기술이면 만들 수 있다고 봤습니다. 검색도 되고 에이전트도 되니 붙이기만 하면 된다고 본 것입니다.

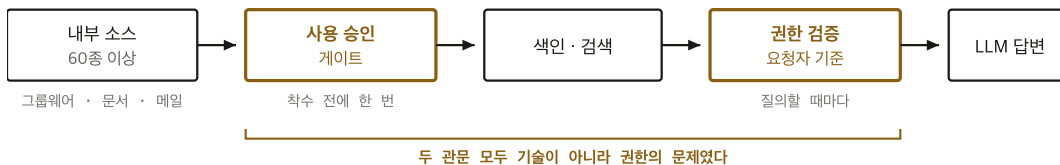
착수하고 나서 알았습니다. **병목이 기술이 아니었습니다**. 내부 문서·메일·그룹웨어는 전부 접근 통제 대상이라, AI가 읽으려면 먼저 사용 승인을 받아야 했습니다. 파이프라인을 아무리 잘 만들어도 데이터에 닿지 못하면 의미가 없었습니다.

판단

그래서 순서를 바꿨습니다. **설계보다 승인을 먼저** 받기로 했습니다. 쓸 수 없는 데이터를 전제로 구조를 짜면 두 번 일하게 됩니다.



운영 구조



파이프라인을 아무리 잘 만들어도 데이터에 닿지 못하면 0점이다

전사 사용 · 승인받은 데이터 소스 60종 이상 · 프로덕션 운영 중

순서를 바꾼 것이 설계의 전부입니다. 승인은 **착수 전 한 번**, 권한 검증은 **질의할 때마다** — 같은 "권한"이지만 놓이는 자리가 다릅니다.

구현

필요한 데이터 범위·사용 목적·통제 방식 정리 → **사용 승인 확보**

→ 확보한 자원 기준으로 검색·권한·모델 호출 아키텍처 설계

→ 그룹웨어 · 메일 · 전자결재 연동과 답변 근거 인용까지 직접 구현

단순 질의응답이 아니라 내부 자원을 통합 활용하는 업무 지원 AI로 발전했고 현재 프로덕션 운영 중입니다.

기획 단계부터 설계, PoC, 고도화까지 전 과정을 총괄했다.

2025년 인사평가 기록

배운 점

기술 외적 제약이 진짜 병목일 수 있다는 것. 그리고 그 제약은 코드가 아니라 설득으로 푼다는 것.

지금은 어떤 과제든 착수 전에 "이 데이터에 닿을 수 있는가, 없다면 누가 열어줄 수 있는가"부터 확인합니다.

그리고 문제가 하나 더 보였습니다. 서비스마다 인증·배포를 매번 새로 만들고 있었습니다.

Chapter 5

반복 가능하게 만들었다

사내 AI 플랫폼 · 2025 – 현재 · 설계·구현

스스로 정의한 문제

PLATFORM

문제

AI 서비스가 늘수록 같은 일을 반복하고 있었습니다. 인증·인가 방식이 서비스마다 제각각이었고, 새 과제를 시작할 때마다 초기 설계를 처음부터 다시 했습니다.

처음 가설과 그 한계

공용 라이브러리를 만들어 각 서비스가 가져다 쓰게 하면 될 거라 봤습니다. 가장 손이 덜 가는 방법이었습니다.

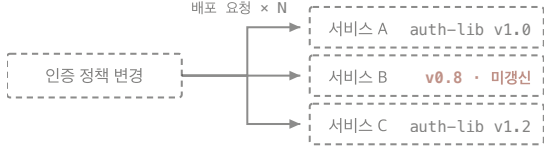
그런데 라이브러리는 각 서비스가 버전을 올려줘야 반영됩니다. 인증 정책이 바뀔 때마다 모든 서비스에 배포를 요청해야 했고, 실제로는 아무도 제때 올리지 않았습니다. 공통화했는데 여전히 제각각이었습니다.

판단

인증·인가를 각 서비스 안에 두는 한 같은 문제가 반복된다고 봤습니다. 코드로 공유하지 말고 경로로 강제해야 한다는 판단이었습니다.

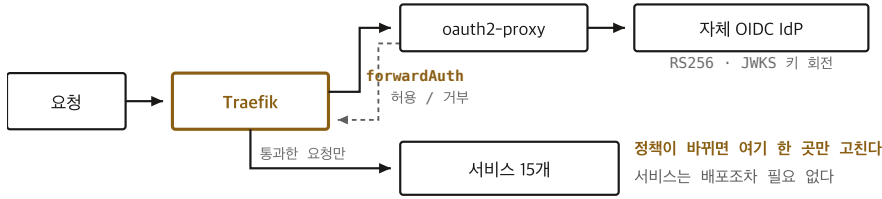
그래서 게이트웨이 앞단에 몰았습니다. 서비스가 아무것도 하지 않아도 요청이 반드시 인가 계층을 지나게 만드는 구조 — forwardAuth를 고른 이유입니다. 정책이 바뀌어도 게이트웨이만 고치면 되고, 서비스는 배포조차 필요 없습니다.

처음 - 라이브러리로 공통화 (가져다 쓰게 한다)
배포 요청 × N



각 서비스가 버전을 올려야 반영된다
→ 아무도 제때 올리지 않았다

바꾼 구조 - 경로로 강제 (반드시 지나가게 한다)



서비스가 아무것도 하지 않아도 요청은 인가 계층을 지난다 - 지키는 사람이 아니라 구조가 보장한다

같은 "공통화"인데 강제력이 다릅니다. 위는 각 서비스가 지켜줘야 성립하고, 아래는 안 지킬 방법이 없습니다.

구현

Traefik + oauth2-proxy에 **자체 OIDC IdP**를 붙이고 forwardAuth로 인가를 강제했습니다. RS256 / JWKS 키 회전 체계를 두고, 내부는 Clean Architecture 4계층 + DDD로 경계를 정리했습니다. 같은 판단을 다음 과제가 반복하지 않도록 공통 구조를 **재사용 서비스 템플릿**으로 추출했고, 기획·프론트엔드 담당자와 파트별 경계를 합의했습니다.

결과

수용 서비스

15개

게이트웨이 뒤에서 가동 중

설계 판단 기록

349건

ADR · 설계문서

배포 범위

타 조직

다른 그룹·연구소로 확산

템플릿 파생

4+

공통 구조 재사용

리드 인원

7명

AI 파트 · 파트별 경계 합의

배운 점

공통화에는 두 가지가 있다는 걸 알았습니다. **가져다 쓰게 하는 것**과 **지나가게 하는 것**. 전자는 지키는 사람에게 의존하고, 후자는 구조가 보장합니다.

지금은 공통 규칙을 만들 때 **"안 지키면 어떻게 되는가"**를 먼저 생각합니다. 안 지켜도 아무 일 없으면 그런 규칙이 아니라 권고입니다.

그런데 플랫폼이 커지자 다른 것이 부족해졌습니다. **돌릴 자원이 모자랐습니다.**

Chapter 6

안정적으로 운영했다

전사 모델 서빙 환경 · 2026.06 – 현재 · 설계·운영

스스로 정의한 문제

INFRA

문제

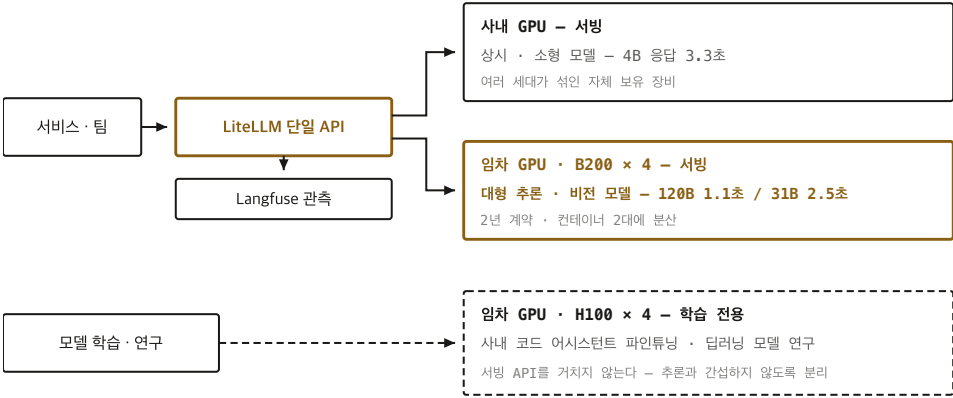
팀마다 모델을 띄우려 했지만 GPU는 한정돼 있었습니다. 외부 GPU를 2년 계약으로 임차했는데, 사내 장비와 어떻게 나눠 쓸지 **아무 기준이 없었습니다.**

판단

"무엇을 어디에 올릴 것인가"는 **측정 없이 정할 수 없는 문제**라고 봤습니다. 감으로 배치하면 나중에 근거를 댈 수 없습니다.

동일 조건(입력 1024 / 출력 256 토큰)으로 같은 모델을 양쪽에서 재고, 그 표를 배치 기준으로 삼기로 했습니다.

무엇을 어디에 올릴지는 재고 나서 정했다



벤치마크 80회 (모델 8종 × 조건 10회) · 동일 조건 입력 1,024 / 출력 256 토큰

어느 모델이 어디로 가는지는 **측정이 정했습니다**. 서버는 단일 API 뒤로 묶어 호출하는 쪽이 어디에 떠 있는지 몰라도 되게 했고, **학습은 서버 경로 밖으로 빼** 긴 작업이 실시간 응답을 건드리지 않게 했습니다.

결과

상시 서버 모델 8종 단일 API 통합	컨텍스트 상한 4× 추가 메모리 0으로	벤치마크 80회 8종 × 조건 10회
운영 GPU 19대 서빙 15 · 학습 전용 4	최대 격차 5.7× 비전·한국어 31B	작업 기록 17건 판단 근거까지 표준화

동일 조건 벤치마크 · 입력 1,024 / 출력 256 토큰 · 단일 요청 응답 시간

모델 규모	사내	임차	차이
대형 추론 (120B)	5.2초	1.1초	4.7×
비전·한국어 (31B)	14초	2.5초	5.7×
소형 고속 (4B)	3.3초	0.8초	4.1×

비전·한국어 모델이 사내에서 14초가 걸린다는 사실은 재기 전까지 아무도 몰랐습니다. 이 표를 경영진 리포트로 만들어 **기술 결정이 아니라 자원 배분 결정**으로 올렸습니다.

내가 만든 장애

임베딩 모델을 추가하다 한 GPU를 과구독시켜 **같은 장비의 다른 모델을 죽였습니다**. 기록에 원인을 "내가 만든 빔"으로 적었습니다.

가장 쉬운 복구는 추가한 모델을 지우는 것이었지만, 점유 현황을 전수 조사해 소형 모델 하나가 자기 크기의 6배가 넘는 캐시를 잡고 있는 것을 찾았습니다. 그것을 줄여 **체감 손실 없이 회수**했습니다. 고치는 과정에서 한 번 더 실패한 설정값까지 기록에 남겼습니다.

이 사건은 "모델 추가 시 GPU당 안전마진을 남기고 예산표를 갱신한다"는 규칙으로 바뀌었습니다.

그때 GPU를 안다고 생각했는데, 아직 아니었다는 걸 알았습니다. 메모리 설정값이 무엇을 조절하고 무엇을 조절하지 않는지조차 정확히 모른 채 숫자만 만지고 있었습니다.

배운 점

측정하지 않으면 결정할 수 없고, 기록하지 않으면 반복합니다. 작업 기록 17건을 증상→가설→증거→판정→조치→교훈 구조로 표준화한 이유입니다.

지금은 자원을 건드리기 전에 무엇이 무엇을 바꾸는지부터 확인합니다. 설정값 하나를 내리기 전에 그것이 실제로 어느 항목을 줄이는지 먼저 채고, 틀렸던 값도 함께 기록에 남깁니다.

HOW I WORK

제시된 가설을 데이터로 검증합니다

원인을 특정해 전달받더라도 그대로 채택하지 않고, 측정으로 먼저 반증합니다.

신고된 가설	실제 원인	판정 근거
"게이트웨이 타임아웃"	정상적인 토큰 상한 소진 + 호출측 라우팅 오류	출력 토큰이 요청 상한과 정확히 일치. 응답 마지막 바이트를 상대측 캡처의 절단 지점과 글자 단위로 대조
"프로세스 데드락"	큐 포화 (대기 319건)	health 응답 정상, 로그 실시간 갱신, GPU 오류 전무
"기동 실패"	JIT 컴파일 정상 동작 + 별개 3종 원인	전체 프로세스 트리 확인 — 컴파일러가 하위 깊이에서 동작 중

첫 번째 사례에서는 요청받지 않은 문제까지 발견해 돌려줬습니다. 타임아웃 여부만 확인해 달라는 요청이었지만, 조사 과정에서 **요청이 의도한 모델이 아닌 다른 모델로 라우팅되고 있던 별개 결함**이 드러났습니다.

틀린 것을 남깁니다

제 실수로 생긴 장애도 원인을 명시해 기록하고, 고치다 실패한 설정값까지 적어 재발 방지 규칙으로 정리합니다. 다음 사람이 같은 곳을 밟지 않게 하기 위해서입니다.

단순 작업 로그가 아니라 "왜 그렇게 판단했는가"를 남기는 게 목적.
작업 기록 문서 서문 중

알아낸 것을 조직에 옮깁니다

혼자 알고 있으면 그 판단은 제가 자리를 비우는 순간 사라집니다. 2026년 사내 재직자 교육 **21개 과정 중 AI 과정 2개**에 강사로 배정돼 커리큘럼부터 교재까지 **직접 설계하고 강의**했습니다. 강사진은 대부분 부장·수석부장·임원 직급이었고, 저는 대리였습니다.

과정	분량	구성
LLM · RAG 기반 AI 활용	3일 27시간	슬라이드 155장 · 실습 14회 를 이론과 교차 배치
AI 트렌드 및 사내 솔루션 이해	2시간	슬라이드 94장 · 기술 동향 · 사내 적용 사례 · 기술 선택 기준

커리큘럼을 "무엇을 할 수 있다"가 아니라 "무엇이 안 되는가"로 짰습니다.

프롬프트의 한계 → **RAG** → Naïve RAG의 한계 → **Function Calling**
→ 그 한계 → **멀티 도구 에이전트** → **AI Agent**

앞 단계에서 막히는 지점이 다음 단계의 이유가 되게 했습니다. 이 문서의 구조와 같습니다. 마지막 장은 기술 설명이 아니라 **판단 기준**이었습니다 — 상용이나 온프레미스냐, 어시스턴트냐 에이전트냐를 사업 요구사항별로 표로 정리했습니다. 5·6장에서 제가 세운 기준을 조직이 꺼내 쓸 수 있는 형태로 옮긴 것입니다.

담당 과정 2 / 21 사내 전 과정 중 AI 과정 전부	교육 시간 27시간 21개 과정 중 최장 · 3일 연속	제작 슬라이드 249장 두 과정 합계 · 전량 직접 제작
실습 14회 이론과 교차 배치		

수강 후 설문에서 두 과정 모두 **기대충족도는 응답자 전원이 "매우 만족"**이었습니다.

LLM / RAG / Agent의 차이 및 특징을 확실히 구분할 수 있게 되었습니다.
수강 후 설문 중

팀과 함께 일합니다

현재 AI 파트 리드로 과제 우선순위와 기술 방향을 정하고 구현을 분배하되, **핵심 설계와 검증은 직접 맡습니다.**
관리로 빠지지 않는 것을 전제로 일해 왔습니다. 다른 파트와는 기술 경계와 인터페이스 기준을 먼저 합의해,
서로 기다리지 않고 진행할 수 있게 만듭니다.

지금 하고 있는 것

앞의 여섯 장은 끝난 일이고, 아래는 진행 중입니다. **역할이 서로 다릅니다** — 직접 구현한 것과 방향만 잡은
것을 구분해 적었습니다.

과제	내용	내 역할
플랫폼 외부 확산	사내 다른 그룹·연구소로 배포, 공공 SI 사업 1건에 적용 중	직접 구현
보안악점 자동 진단 AI	정적 분석·의미 검색·LLM 탐지를 병렬로 돌리고 오탐을 교차 검증해 공공 SI 진단 의무에 대응	기획 · 설계 · 리드
제안서 검색 시스템	담당자 기억과 인적 네트워크에 의존하던 과거 제안서 탐색을 의미 검색으로 대체. 이후 채팅형 생성까지 확장	발의 · 기획 · 리드
사내 코드 어시스턴트 모델	외부 클라우드 도구를 쓸 수 없는 환경이라 온프레미스 자체 모델로. 사내 코드에서 합성 데이터셋을 만들어 파인튜닝	기획 · 설계 · 데이터 준비 · 리드

세 번째가 **1장으로 돌아옵니다.** 코드 어시스턴트도 결국 **학습 데이터가 없는 문제**였습니다. 사내 코드는 있지만
그대로는 학습 데이터가 아니어서, 무엇을 어떤 형태로 만들어야 모델이 배울 수 있는지부터 정해야 했습니다.
3년 전 고문서에서 208자를 만들어 넣던 것과 같은 문제입니다.

다음에 풀고 싶은 문제

여섯 장을 지나며 다루는 단위가 모델에서 기반으로 옮겨 왔습니다. 그런데 3장에서 세운 "확률적인 것을 어디에 가둘 것인가"라는 기준은 지금 에이전트 하나 안에서만 성립합니다.

에이전트가 여러 개 붙고 같은 자원·같은 권한 체계를 공유하기 시작하면, 실패가 어느 에이전트의 어느 Tool에서 났는지를 지금 방식으로는 특정할 수 없습니다. 여기까지가 제가 아직 답을 내지 못한 지점입니다. 다음에는 이 문제를 검증 가능한 형태로 바꾸는 일부터 해보고 싶습니다.

기술 스택

여섯 장을 지나며 직접 쓰고 운영해 온 범위입니다.

모델 · 데이터	PyTorch · 탐지/인식 모델 튜닝 · 데이터 증강 생성 모델 · 데이터셋 구축·레이블링
LLM · 에이전트	LangChain · LangGraph · Advanced RAG · Tool Calling · 프롬프트 전략
백엔드 · 플랫폼	Python · TypeScript · FastAPI · OIDC · Traefik · Clean Architecture + DDD PostgreSQL · PostGIS · Vector DB · Redis
서빙 · 운영	vLLM · LiteLLM · 관측/트레이싱 · GPU 자원 배분 · Docker · CI/CD

회사 소스 코드와 내부 식별 정보는 포함하지 않았습니다. 수치는 사내 문서에 기록된 측정값이며 재현 절차가 함께 남아 있습니다. 직접 구현한 작업과 방향만 제시한 작업을 구분해 표기했습니다.